



Taller de Ejercicios

Guía Práctica de Docker

Christian Ramirez
@christian_ramirezz



@latecnologiaavanza

Ejercicio #1

Se necesita levantar un servidor web Nginx usando Docker, accesible desde el navegador, sin afectar el host, y poder gestionar su contenedor fácilmente: crear y ejecutar el contenedor con nombre y puerto mapeado, verificar que esté activo, acceder desde el navegador, detenerlo y finalmente eliminarlo. El resultado esperado es que Nginx funcione correctamente, sea accesible y el contenedor pueda detenerse y eliminarse sin problemas.



Ejercicio #2

Se necesita aprender a usar contenedores Docker de manera interactiva para ejecutar comandos básicos en un entorno Linux sin afectar el sistema host. Para ello, inicia un contenedor Ubuntu 20.04 en modo interactivo, actualiza la lista de paquetes, instala un editor de texto, crea un archivo llamado hola.txt con contenido dentro del contenedor, verifica su contenido y finalmente cierra y elimina el contenedor. El resultado esperado es que todos los comandos se ejecuten correctamente dentro del contenedor, el archivo se cree con éxito y el contenedor se elimine sin dejar residuos en el host



Ejercicio #3

Se necesita crear una imagen Docker que ejecute automáticamente un script de Python, de manera que el código pueda correr dentro de un contenedor sin depender del entorno del host. Para ello, crea un script llamado `saludo.py` que imprima el mensaje “¡Mi primer script en Docker!”, define un `Dockerfile` que use una imagen base de Python, copie el script al contenedor y lo ejecute al iniciar, construye la imagen Docker y ejecuta un contenedor a partir de ella. El resultado esperado es que al correr el contenedor, se muestre el mensaje del script en la consola y el contenedor se elimine automáticamente si se utiliza la opción correspondiente, demostrando que el contenedor ejecuta código Python de manera independiente y reproducible



Ejercicio #4

Se necesita desplegar una aplicación web Flask dentro de un contenedor Docker, accesible desde el navegador en el puerto 5000, incluyendo todas sus dependencias sin afectar el host. Para ello, crea un contenedor usando la imagen base de Python 3.9 slim, configura el directorio de trabajo, copia e instala las dependencias listadas en requirements.txt, agrega el código de la aplicación Flask, expón el puerto correspondiente y define el comando de ejecución. El resultado esperado es que al acceder desde el navegador a la dirección del host y puerto asignado, se muestre el mensaje “¡Hola, mundo desde un contenedor Docker!” y que el contenedor pueda ejecutarse, detenerse y eliminarse correctamente.



Ejercicio #5

Se necesita desplegar una aplicación web FastAPI dentro de un contenedor Docker, accesible desde el navegador o mediante clientes HTTP en el puerto 8000, incluyendo todas sus dependencias sin modificar el host. Para ello, crea un contenedor usando la imagen base de Python 3.9 slim, configura el directorio de trabajo, copia e instala las dependencias listadas en requirements.txt, agrega el código de la aplicación FastAPI, expón el puerto correspondiente y define el comando de ejecución con Uvicorn. El resultado esperado es que al acceder a la ruta raíz de la aplicación se reciba la respuesta `{"message": "Hola mundo"}` y que el contenedor pueda iniciarse, detenerse y eliminarse correctamente



Ejercicio #6

Se necesita trabajar con volúmenes en Docker para almacenar datos de manera persistente aunque los contenedores se eliminen. Para ello, crea un contenedor de Ubuntu con un volumen, guarda un archivo dentro del volumen, cierra y elimina el contenedor, y luego crea un nuevo contenedor reutilizando el mismo volumen para verificar que el archivo persiste. El resultado esperado es que los datos guardados en el volumen permanezcan accesibles incluso después de eliminar el primer contenedor



Ejercicio #7

Se necesita asegurar la persistencia de datos en una base de datos MySQL ejecutada en Docker, de manera que la información no se pierda al detener o eliminar el contenedor. Para resolverlo, crea un volumen para almacenar los datos de MySQL, ejecuta un contenedor con la imagen oficial de MySQL montando ese volumen y definiendo la contraseña del usuario root mediante una variable de entorno, y luego verifica que al detener y eliminar el contenedor, al recrearlo los datos, tablas y registros sigan intactos. El resultado esperado es que las bases de datos permanezcan accesibles y consistentes mientras el volumen exista, garantizando persistencia de la información



Ejercicio #8

Se necesita desplegar una aplicación backend que se conecte a una base de datos PostgreSQL en Docker, simulando un entorno de red privada de manera que los contenedores puedan comunicarse entre sí usando nombres lógicos. Para resolverlo, crea una red Docker privada, lanza un contenedor con PostgreSQL conectado a esa red y con la contraseña del usuario configurada, y luego verifica la conectividad creando un contenedor temporal en la misma red que haga ping al contenedor de PostgreSQL. El resultado esperado es que el nombre del contenedor de la base de datos se resuelva automáticamente dentro de la red y que se pueda comunicar, demostrando que el DNS interno de Docker funciona correctamente



Ejercicio #9

Se necesita desplegar una aplicación web completa usando Docker Compose, integrando un backend en Flask y un frontend Nginx como proxy inverso, de manera que los contenedores se comuniquen entre sí automáticamente y el frontend reciba y reenvíe las solicitudes al backend. Para resolverlo, configura un proyecto con la estructura adecuada, crea el backend con Flask escuchando en el puerto 5000, define un Dockerfile para instalar dependencias y ejecutar la app, configura Nginx para que actúe como proxy inverso apuntando al backend, y define un archivo docker-compose.yml que levante ambos servicios, asegure que el backend se inicie antes que el frontend y exponga el puerto 80 al host. El resultado esperado es que al ejecutar `docker compose up`, se pueda acceder desde el navegador a `http://localhost` y la respuesta provenga correctamente del backend Flask a través de Nginx, demostrando comunicación entre contenedores y orquestación completa con Docker Compose.



Ejercicio #10

Se necesita desplegar una aplicación web Flask que se conecte a un servicio Redis utilizando Docker Compose, de manera que ambos contenedores puedan comunicarse automáticamente y persistir los datos del contador de visitas. Para resolverlo, define los servicios en un archivo `docker-compose.yml`, construye el contenedor de Flask a partir de una imagen base de Python con sus dependencias instaladas, expón el puerto 5000 y configura las variables de entorno necesarias para conectarse a Redis; lanza un contenedor Redis con un volumen para guardar los datos y asegúrate de que Flask dependa de Redis. El resultado esperado es que al acceder a la aplicación desde el navegador, se incremente un contador de visitas almacenado en Redis y que ambos contenedores puedan iniciarse, detenerse y eliminarse correctamente mediante Docker Compose, demostrando comunicación entre servicios y persistencia de datos



Ejercicio #11

Se necesita crear un entorno de desarrollo profesional para una aplicación Node.js utilizando Docker y Nodemon, de manera que el código se ejecute dentro del contenedor, los cambios locales se reflejen automáticamente (hot reload) y no sea necesario instalar Node ni dependencias globales en el host. Para resolverlo, define un servicio en Docker Compose usando la imagen de Node.js, monta el código local como volumen, asegura que `node_modules` se mantenga dentro del contenedor, expón el puerto 3000 y ejecuta el comando que instale dependencias y arranque Nodemon. El resultado esperado es que al acceder a `http://localhost:3000` se vea el mensaje de bienvenida, cualquier cambio en el archivo `server.js` reinicie automáticamente el servidor dentro del contenedor y todo funcione sin afectar el sistema local.



Ejercicio #12

Se necesita automatizar el proceso de construcción y despliegue de una aplicación Node.js usando Docker en un pipeline CI/CD con GitHub Actions y Docker Hub, de manera que la imagen resultante sea inmutable y reproducible, y se pueda ejecutar en cualquier máquina sin instalar dependencias locales. Para resolverlo, crea un proyecto mínimo de Node.js con un servidor básico, define un Dockerfile que construya la imagen con todas las dependencias, sube el proyecto a GitHub y configura un workflow que haga checkout del código, loguee en Docker Hub usando secrets, construya la imagen y la publique en tu repositorio de Docker Hub. El resultado esperado es que tras hacer un push a la rama principal, la acción CI se ejecute correctamente, la imagen se suba al repositorio y al ejecutarla en cualquier máquina se muestre el mensaje del servidor, demostrando un flujo completo de CI/CD con Docker.



Ejercicio #13

Se requiere crear un contenedor Docker ultraligero para una aplicación escrita en C, de manera que el ejecutable final se ejecute sin necesidad de incluir herramientas de compilación ni librerías adicionales en la imagen de producción. Para resolverlo, utiliza un Dockerfile con multi-stage builds, creando primero una etapa de compilación con gcc que genere un binario estático, y luego una etapa final mínima basada en scratch que solo contenga el ejecutable. El resultado esperado es que, al construir y ejecutar la imagen, el contenedor muestre correctamente el mensaje de la aplicación C en consola, y que la imagen final tenga un tamaño drásticamente menor que la imagen completa de compilación, demostrando la eficiencia de separar build y runtime.



Nuestra comunidad



@latecnologiaavanza



latecnologiaavanza



@latecnologiaavanza



La Tecnología Avanza (Comunidad)

Suscríbete



“Empieza a aprender ahora porque la tecnología avanza y tú también”